

Technical Report: Implementation of a Minimal Debugger

Course Code: COD7001

Mayur Kamble 2025MCS2116

Nizaul Rahman 2025MCS2099

1 Introduction and Project Goal

This report details the design and implementation of a minimal debugger, `minigdb`, developed for ELF binaries on a Linux platform. The project's primary goal was to implement core debugging functionalities—loading, execution control, breakpoint management, and inspection—using operating-system-level primitives, specifically the `ptrace` system call, without reliance on standard debugging tools like GDB.

The implemented debugger successfully met all Mandatory Functional Requirements (Load/Execute, Breakpoints, Step/Continue, Register Inspection, Process Status) and incorporated the following Optional Extensions:

- **Memory Inspection:** Ability to examine the debuggee's memory at an arbitrary address.
- **Shell Integration:** Improved command parsing for a more flexible user interface.

2 Architectural Design: The PTRACE Model

The debugger operates using the standard parent-child debugging model, leveraging the `ptrace` system call.

2.1 Process Setup (Fork/Execve)

The debugger program executes the following sequence to initiate tracing:

1. **Fork:** The debugger (`minigdb`) calls `fork()` to create a child process. The parent becomes the tracer, and the child becomes the tracee.

2. **PTRACE_TRACEME**: In the child process, `ptrace(PTRACE_TRACEME, 0, NULL, NULL)` is called. This informs the kernel that the parent will trace the child after `execve`.
3. **Execve**: The child loads and begins executing the target binary by calling `exec1()`. Immediately after `execve`, the kernel halts the child and sends `SIGTRAP` to the parent.
4. **Waitpid**: The parent calls `waitpid()` to wait for the initial stop signal, confirming debugging control.

This ensures the debugger has full control from the first executed instruction.

3 Core Mechanism: Software Breakpoints

Execution control is implemented using software breakpoints through `PTRACE_PEEKTEXT` and `PTRACE_POKETEXT`.

3.1 Breakpoint Placement (The 0xCC Trap)

A software breakpoint replaces the first byte of a target instruction with the `INT3` opcode (`0xCC`).

The `enable_breakpoint` procedure:

1. **Read Original Word**: Using `PTRACE_PEEKTEXT`.
2. **Save Original Byte**: Stored inside a breakpoint structure.
3. **Write Trap Byte**: Replace first byte with `0xCC` using `PTRACE_POKETEXT`.

3.2 Breakpoint Handling and Step-Over

When the tracee executes the `0xCC`, the kernel stops it and sends `SIGTRAP`.

Handler logic:

1. **Adjust RIP**: `RIP` points past the trap; it is decremented by one.
2. **Restore Instruction**: Original byte restored with `POKETEXT`.
3. **Single-Step**: Execute one instruction.
4. **Reinstall Breakpoint**: Write back `0xCC`.

4 Process Control and Inspection Primitives

4.1 Execution Control

- **Continue:** `PTRACE_CONT` resumes execution.
- **Single-Step:** `PTRACE_SINGLESTEP` executes one instruction.

4.2 Inspection

Register Inspection: `PTRACE_GETREGS` fetches all general-purpose registers.

Process Status: Using `waitpid()` macros:

- `WIFSTOPPED()`
- `WIFEXITED()`
- `WSTOPSIG()`

5 Handling Position-Independent Executables (PIE/ASLR)

Modern Linux systems use ASLR, causing executables to load at random addresses.

Symbol table addresses such as `0x11aa` are Relative Virtual Addresses (RVAs).

To compute actual breakpoint locations:

1. Stop the process immediately after `execve`.
2. Read memory layout from: `/proc/<PID>/maps`.
3. Extract the base address of the binary's `.text` segment.

Absolute address calculation:

$$\text{Absolute Address} = \text{Base Address} + \text{RVA}.$$

This prevents `PTRACE_PEEKTEXT` from failing with I/O errors due to invalid addresses under ASLR.

6 Optional Extensions

6.1 Memory Inspection

The debugger supports:

- **PTRACE_PEEKDATA:** Reads 8-byte words.
- **Iterative Memory Dumping:** Repeated peeks to dump user-specified byte counts.

6.2 Shell Integration

Enhancements include:

- Trimming whitespace, normalizing input.
- Parsing complex commands like `x /<count> 0xADDRESS`.

7 Conclusion

The `minigdb` project successfully implements a functional debugger built entirely on `ptrace`. It demonstrates mastery of low-level process control, memory manipulation, and register inspection. Additionally, PIE/ASLR support ensures correct behavior on modern Linux systems. Memory inspection and improved shell parsing contribute to a more complete and practical debugging tool.